DESIGN AND IMPLEMENTATION OF A 4-MODE SPI MASTER CONTROLLER USING VERILOG

A Project Report

by

Smaran Yanapu

Department of Electronics and Communication Engineering

Birla Institute of Technology, Mesra

TABLE OF CONTENTS

ABSTRACT 4
INTRODUCTION 5
Background of SPI Communication Protocol 5
Importance and Applications of SPI in Electronics 5
Objective of the Project 6
Scope and Limitations of the Project 6
LITERATURE REVIEW 7
Previous Work on SPI Communication Hardware 7
Identified Gaps and Motivation for the Present Work 7
METHODOLOGY 8
Tools and Software Used 8
Design Approach 8
Implementation Details 11
Verilog Coding Practices 11
SPI Master Control Module (FSM) 11
SPI Clock Generation and Synchronization 12
Data Transmission and Reception Logic 12
Clock Edge Detection and Mode Control 12
Chip Select and Transaction Control 13
RTL Schematic and Structural Verification 13
Testing and Validation 16
Verification Methodology 16
SPI Slave Model for Verification 16
Test Cases and Simulation Strategy 16
Waveform Analysis and Validation Results 17
Tools Used for Verification 17
RESULTS 18
Simulation Results and Console Output 18
Timing Diagram and Waveform Analysis 18
CHALLENGES AND SOLUTIONS 20
Correct Handling of CPOL and CPHA Across All SPI Modes 20

CPHA Preload Handling and First-Bit Alignment 20
SPI Clock Generation and Idle State Control 20
Bit Count Control and Transaction Completion 21
Verification Across All SPI Modes 21
CONCLUSION 22
REFERENCES 23
APPENDICES 24
SPI Master RTL Code 24
SPI Master Testbench Code 30

ABSTRACT

This project presents the design and implementation of a 4-mode SPI (Serial Peripheral Interface) Master controller using Verilog HDL. The objective of the project is to develop a protocol-accurate SPI Master capable of supporting all four SPI modes defined by clock polarity (CPOL) and clock phase (CPHA), and to verify its correct timing behavior through simulation.

The design is based on a finite state machine (FSM) architecture that controls data transmission, reception, and clock generation. Mode-dependent shift and sample edge logic is implemented to ensure correct data transfer for each CPOL/CPHA combination. An internal clock divider is used to generate the SPI clock with proper idle behavior.

Verification is carried out using a self-checking testbench that includes a behavioral SPI slave model. All four SPI modes are simulated and validated through bidirectional data checking, waveform-based timing analysis, and review of RTL analysis and synthesis schematics. The results confirm correct protocol operation and timing accuracy for single-byte full-duplex SPI communication.

INTRODUCTION

Background of SPI Communication Protocol

The Serial Peripheral Interface (SPI) is a synchronous serial communication protocol widely used for short-distance communication between digital integrated circuits. SPI operates on a master–slave architecture, where the master device controls the communication by generating the clock signal and selecting the slave device using a chip select signal.

SPI communication typically uses four signals: Serial Clock (SCLK), Master Out Slave In (MOSI), Master In Slave Out (MISO), and Chip Select (CS). Data transfer in SPI is full-duplex, allowing simultaneous transmission and reception of data between the master and the slave. The timing behavior of SPI communication is defined by two parameters: Clock Polarity (CPOL) and Clock Phase (CPHA), which together determine four standard SPI modes.

Importance and Applications of SPI in Electronics

SPI is widely used in electronic systems due to its simple hardware interface, high data transfer rates, and deterministic timing behavior. Unlike asynchronous protocols, SPI does not require complex addressing or handshaking mechanisms, making it suitable for high-speed and low-latency communication.

SPI is commonly used for interfacing microcontrollers with peripherals such as sensors, analog-to-digital converters (ADC), digital-to-analog converters (DAC), memory devices, display controllers, and communication modules. Its simplicity and reliability make it an important protocol in embedded systems and digital hardware design.

Objective of the Project

The primary objective of this project is to design and implement a synthesizable SPI Master controller using Verilog HDL that supports all four SPI modes defined by CPOL and CPHA. The project aims to achieve protocol-accurate timing behavior, correct data transfer, and reliable operation across different SPI configurations.

Another key objective is to verify the functionality and timing correctness of the design through simulation using a self-checking testbench and waveform analysis.

Scope and Limitations of the Project

The scope of this project includes the design and simulation-based verification of a single-master, single-slave SPI communication system supporting single-byte full-duplex data transfer. The design focuses on correct protocol timing, mode-dependent data shifting and sampling, and FSM-based control logic.

The project does not include multi-byte burst transfers, multi-slave arbitration, FIFO buffering, or integration with standard bus interfaces such as AXI or APB. Hardware implementation on an FPGA board is also outside the scope of this work.

LITERATURE REVIEW

Previous Work on SPI Communication Hardware

The SPI communication protocol has been widely studied and implemented in both academic and industrial digital systems. Standard documentation and application notes from semiconductor manufacturers describe SPI as a simple, synchronous, full-duplex protocol commonly implemented using shift registers and control logic.

Several educational and reference designs implement basic SPI controllers using hardware description languages such as Verilog or VHDL. Most of these designs focus on functional data transfer and often support a single SPI mode, typically Mode 0, with fixed clock polarity and phase.

Previous works also demonstrate SPI master designs integrated into microcontroller or FPGA-based systems for interfacing with memory devices and peripheral modules, where simplicity and functional correctness are generally prioritized.

Identified Gaps and Motivation for the Present Work

Although SPI is a widely used protocol, many existing basic implementations do not comprehensively address all four SPI modes defined by clock polarity (CPOL) and clock phase (CPHA). In particular, correct handling of mode-dependent shift and sample edges, as well as CPHA preload behavior, is often simplified or omitted in introductory designs.

Furthermore, several implementations lack detailed verification methodologies, relying primarily on basic simulation without self-checking mechanisms or systematic waveform-based timing analysis. As a result, timing correctness across different SPI modes is not always explicitly validated.

The present work aims to address these gaps by designing a single SPI Master controller that supports all four CPOL/CPHA modes and by emphasizing simulation-driven verification. The project focuses on correct protocol timing behavior, FSM-based control, and

bidirectional data validation using a self-checking testbench, rather than extending functionality through additional features.

METHODOLOGY

Tools and Software Used

The design and verification of the SPI Master controller were carried out using industry-standard hardware description and simulation tools.

Verilog HDL was used to describe the RTL architecture of the SPI Master controller, including control logic, datapath, and timing-related functionality. The design was written in a synthesizable manner to ensure compatibility with digital hardware implementation.

Xilinx Vivado was used as the primary development and simulation environment. Vivado Simulator was employed to perform functional simulation, timing verification, and waveform analysis for all four SPI modes. The tool was also used to generate RTL analysis and synthesis schematics for structural verification of the design.

Standard simulation utilities provided by the tool were used for debugging, signal monitoring, and verification of protocol timing behavior.

Design Approach

The design of the SPI Master controller was carried out in a systematic, step-by-step manner with emphasis on protocol correctness and timing accuracy. The overall approach involved understanding the SPI timing requirements, defining a suitable control architecture, and implementing the design using modular and synthesizable RTL logic.

The first step in the design process was the analysis of the SPI protocol specification, with particular focus on clock polarity (CPOL) and clock phase (CPHA). Based on these parameters, the four standard SPI modes were identified, and the corresponding data shift and sample edge requirements were derived for each mode.

Next, a finite state machine (FSM)–based control architecture was defined to manage the SPI transaction flow. The FSM was designed with distinct states for idle operation, data loading, chip select assertion, data transfer, and transaction completion. This ensured deterministic control of the SPI signals and proper sequencing of operations.

An internal SPI clock (SCLK) generation mechanism was then implemented using a clock divider derived from the system clock. The idle state of the clock was controlled based on the CPOL setting, ensuring correct clock behavior before and after each transaction.

The data path was designed using shift registers for both transmission and reception. Mode-dependent logic was incorporated to control data shifting and sampling on the appropriate clock edges. Special handling was included for CPHA = 0 operation to preload the first data bit before the initial clock edge.

Finally, the control logic, clock generation, and data path were integrated into a complete SPI Master architecture. The overall design was verified for structural correctness using RTL analysis schematics and was prepared for simulation-based validation.

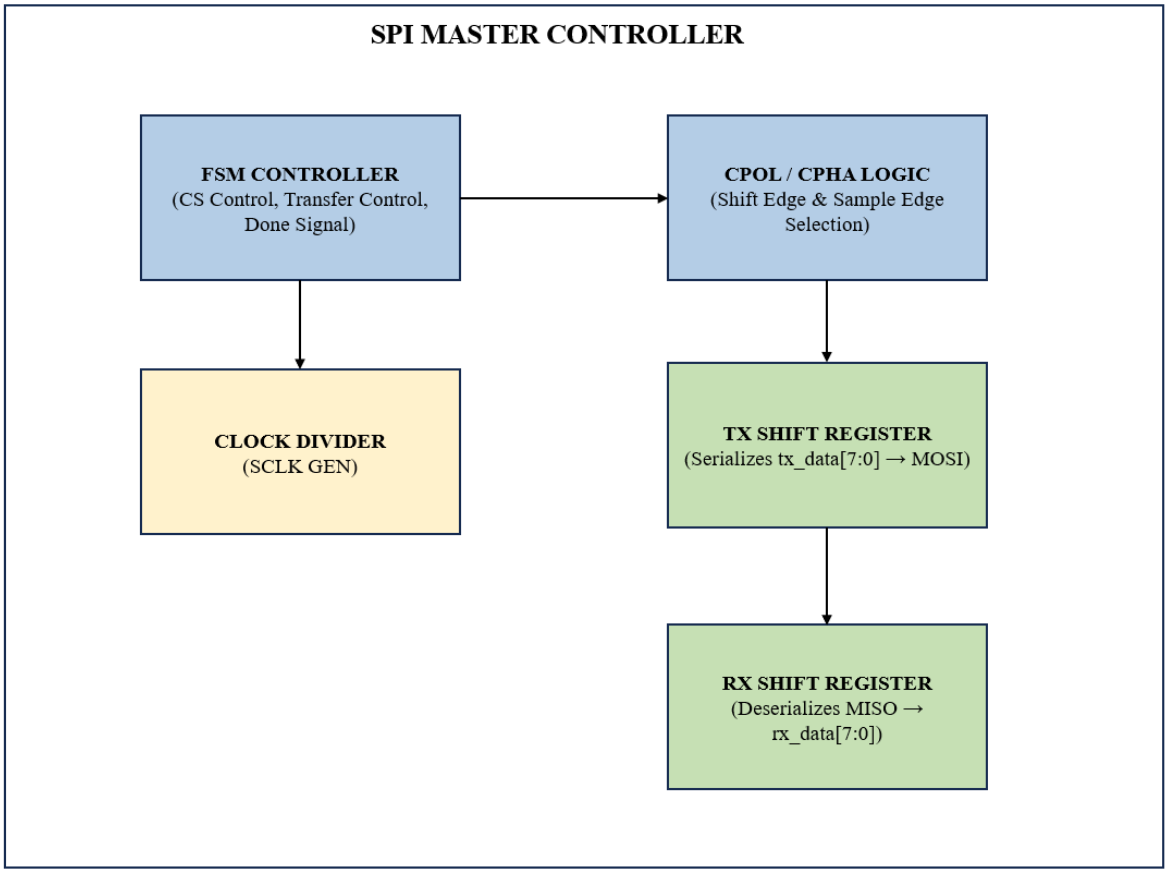


Figure 1. Block Diagram of the SPI Master Architecture

The block diagram illustrates the overall architecture of the SPI Master controller. It consists of an FSM-based control unit, transmit and receive shift registers, SPI clock generation logic, and mode-dependent shift and sample edge control based on CPOL and CPHA. The

architecture separates control and datapath functionality to ensure correct protocol timing and reliable full-duplex data transfer.

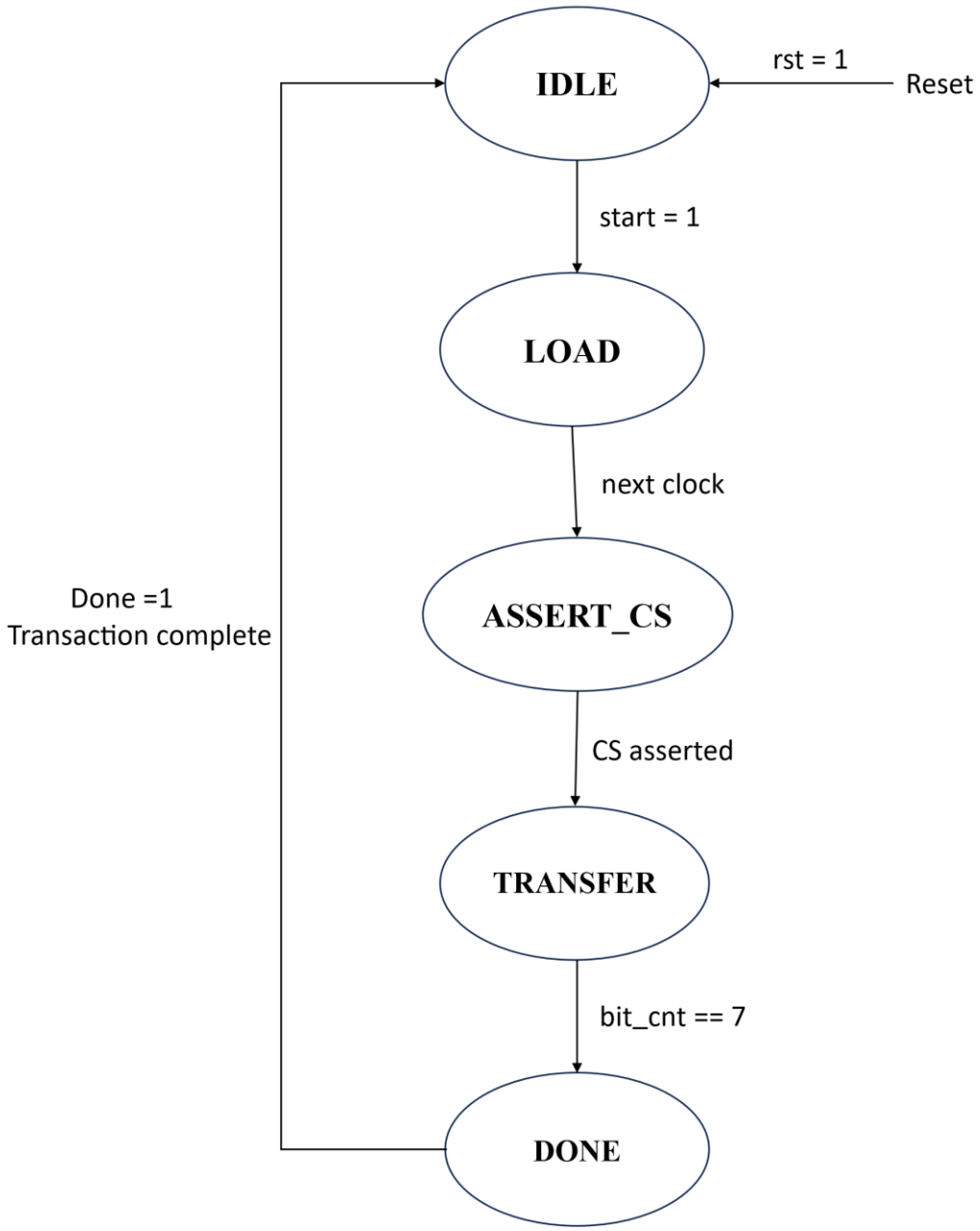


Figure 2. FSM State Diagram of the SPI Master Controller

The FSM state diagram represents the sequential control flow of the SPI Master controller. The controller transitions through idle, data loading, chip select assertion, data transfer, and completion states to manage SPI transactions. This state-based approach ensures deterministic operation, proper synchronization with the SPI clock, and clean termination of each transfer.

Implementation Details

Verilog Coding Practices

The SPI Master controller was implemented using synthesizable Verilog HDL following modular and structured coding practices. The design was developed using synchronous logic with a single system clock and a synchronous reset to ensure predictable and reliable behavior.

Separate always blocks were used for state register updates, next-state logic, clock generation, and data path operations. Blocking and non-blocking assignments were used appropriately to distinguish between combinational and sequential logic. Parameters were used for FSM state encoding to improve code readability and maintainability.

Care was taken to avoid latches by fully specifying signal assignments within each FSM state. The design was written to be compatible with synthesis tools, and all timing-dependent behavior was explicitly controlled through clocked logic.

SPI Master Control Module (FSM)

The core of the SPI Master controller is a finite state machine (FSM) responsible for controlling the SPI transaction sequence. The FSM manages chip select control, data loading, clock enabling, data transfer, and transaction completion.

The FSM consists of the following states:

IDLE, LOAD, ASSERT_CS, TRANSFER, and DONE. In the IDLE state, the SPI bus remains inactive with chip select deasserted. Upon receiving a start signal, the FSM transitions to the LOAD state, where transmit data is latched into the internal shift register.

In the ASSERT_CS state, the chip select signal is asserted, and the SPI clock generation is enabled. The FSM then enters the TRANSFER state, during which serial data transmission and reception occur in synchronization with the SPI clock edges. Once the required number of bits has been transferred, the FSM moves to the DONE state, where the received data is latched and the transaction is completed.

SPI Clock Generation and Synchronization

SPI clock (SCLK) generation is implemented internally using a clock divider derived from the system clock. The divider toggles the SCLK signal at a controlled rate when clock generation is enabled by the FSM.

The idle state of the SCLK signal is determined by the Clock Polarity (CPOL) parameter. When clock generation is disabled, the SCLK signal is forced to the CPOL-defined idle level. This ensures correct clock behavior before and after SPI transactions.

To support mode-dependent data transfer, rising and falling edges of the SCLK signal are detected internally. These detected edges are used to control data shifting and sampling operations based on the selected CPOL and CPHA configuration.

Data Transmission and Reception Logic

Data transmission and reception are implemented using separate shift registers for the MOSI and MISO paths. The transmit shift register holds the data to be sent from the master, while the receive shift register captures data received from the slave.

Mode-dependent logic determines whether data shifting or sampling occurs on the rising or falling edge of the SPI clock. For CPHA = 0 operation, the first transmit data bit is preloaded before the initial clock edge to satisfy SPI timing requirements. For CPHA = 1 operation, data shifting begins after the first clock edge.

A bit counter is used to track the number of bits transferred during a transaction. Once the required number of bits has been transmitted and received, the FSM signals completion of the transfer.

Clock Edge Detection and Mode Control

Clock edge detection logic is implemented internally to support mode-dependent SPI operation. The current and previous values of the SPI clock are compared to detect rising and falling edges. Based on the selected CPOL and CPHA configuration, these detected edges are mapped to shift and sample events.

This mechanism enables precise control of data transmission and reception timing for all four SPI modes, ensuring correct alignment of MOSI and MISO signals with respect to the SPI clock.

Chip Select and Transaction Control

The chip select (CS) signal is controlled directly by the FSM. The CS signal is asserted before the start of data transfer and remains active throughout the transaction. After completion of the transfer, the CS signal is deasserted to terminate communication with the slave device.

This approach ensures clear transaction boundaries and prevents unintended data transfers. The design assumes a single-slave SPI configuration, with one chip select signal managed by the master controller.

RTL Schematic and Structural Verification

The structural correctness of the design was verified using the RTL analysis and synthesis schematics generated by the design tool. The RTL schematic illustrates the hierarchical organization of the SPI Master controller, including the FSM, clock divider, shift registers, and control logic.

Review of the RTL schematic confirms proper connectivity between control and data path elements, absence of unintended combinational loops, and correct signal flow throughout the design. The synthesis schematic further verifies that the design is synthesizable and structurally consistent with the intended architecture.

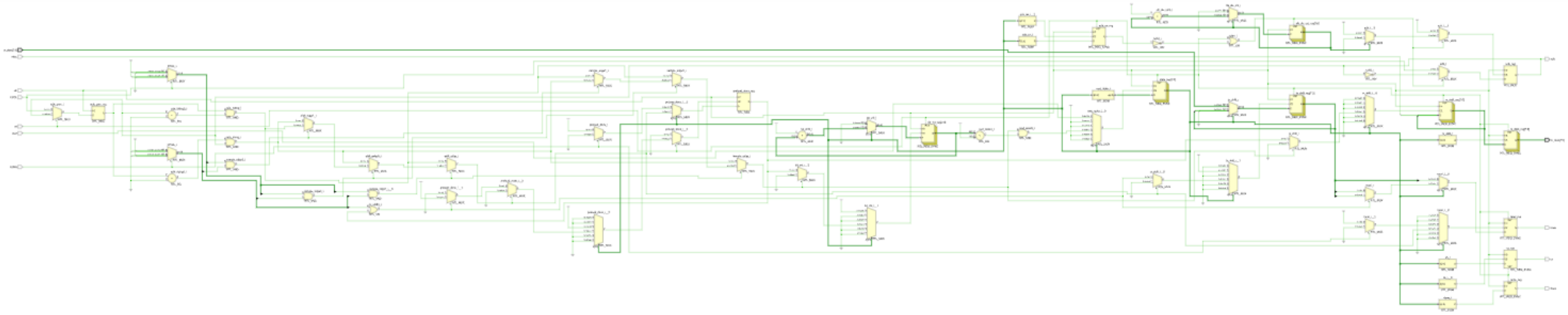


Figure 3. RTL Analysis Schematic of the SPI Master Controller

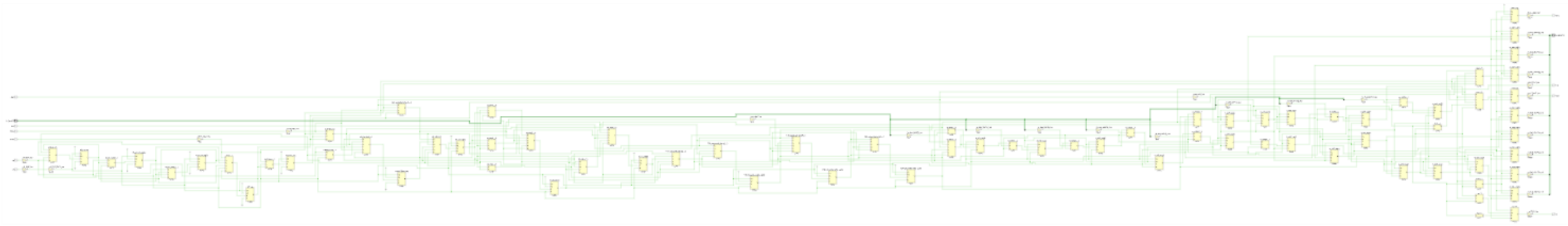


Figure 4. RTL Synthesis Schematic of the SPI Master Controller

Testing and Validation

Verification Methodology

The functionality and timing behavior of the SPI Master controller were verified through simulation using a structured and systematic testbench-based approach. Verification focused on validating correct operation across all four SPI modes defined by CPOL and CPHA, as well as ensuring accurate full-duplex data transfer.

A self-checking testbench was developed to automatically validate the correctness of transmitted and received data. The testbench generates the system clock, applies reset and start conditions, configures CPOL and CPHA settings, and monitors the SPI signals during each transaction. This approach enables repeatable and objective verification without manual inspection of results.

SPI Slave Model for Verification

For verification purposes, a behavioral SPI slave model was implemented within the testbench environment. The slave model responds to the master-generated SPI clock and chip select signals and supports all four SPI modes.

The behavioral slave model provides serial data on the MISO line and samples data from the MOSI line based on the selected CPOL and CPHA configuration. This model is used exclusively for verification and is not part of the synthesizable design. Its purpose is to emulate compliant SPI slave behavior and enable validation of the master controller.

Test Cases and Simulation Strategy

The SPI Master controller was tested using a sequence of test cases covering all four SPI modes. For each mode, a complete SPI transaction was initiated by asserting the start signal, followed by chip select activation and serial data transfer.

During each test case, known transmit data was applied to the master, while predefined data was provided by the slave model. The testbench captured received data at both the master and slave ends. Automatic comparison was performed to verify that the received data matched the expected values.

The test sequence ensured correct chip select behavior, proper SPI clock generation, accurate bit counting, and correct alignment of data shifting and sampling with respect to clock edges.

Waveform Analysis and Validation Results

Simulation waveforms were analyzed to verify correct timing relationships among SPI signals, including SCLK, MOSI, MISO, and CS. Waveform analysis confirmed that data shifting and sampling occurred on the correct clock edges for each SPI mode.

Successful verification was indicated by PASS results generated by the self-checking testbench for all four SPI modes. These results confirm correct protocol operation, timing accuracy, and reliable full-duplex data transfer for single-byte SPI communication.

Tools Used for Verification

Xilinx Vivado Simulator was used to perform functional simulation and waveform analysis. The simulator provided detailed visibility into internal and external signals, enabling verification of control logic, timing behavior, and data transfer mechanisms.

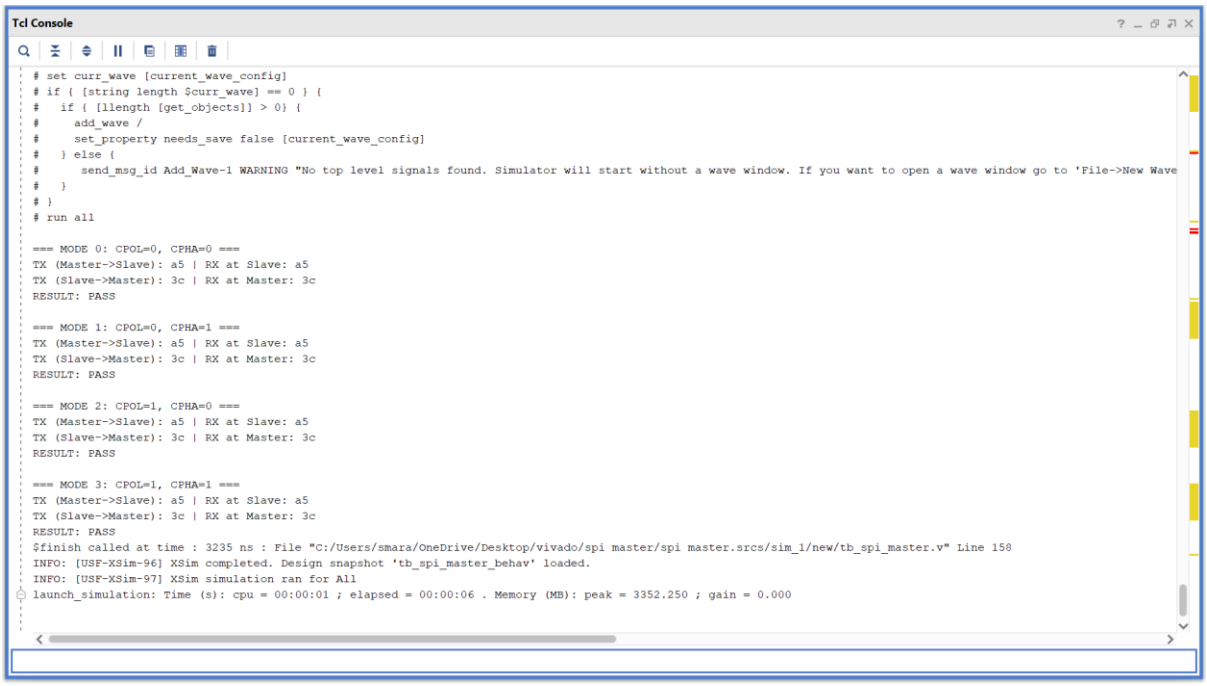
Simulation outputs, including console logs and waveform traces, were used to validate correct operation of the SPI Master controller across all supported modes.

RESULTS

Simulation Results and Console Output

The functionality of the SPI Master controller was verified through simulation using the Vivado Simulator. A self-checking testbench was developed to validate both data transmission and reception between the SPI master and a behavioral SPI slave model.

Simulation console outputs indicate successful completion of SPI transactions for all four SPI modes (Mode 0, Mode 1, Mode 2, and Mode 3). For each mode, the transmitted data from the master was correctly received by the slave, and the data transmitted by the slave was accurately received by the master. The testbench reported a PASS status for all simulated modes, confirming correct functional behavior of the design.



```
Tcl Console
# set curr_wave [current_wave_config]
# if { [string length $curr_wave] == 0 } {
#   if { [llength [get_objects]] > 0} {
#     add_wave /
#     set_property needs_save false [current_wave_config]
#   } else {
#     send_msg_id Add_Wave-1 WARNING "No top level signals found. Simulator will start without a wave window. If you want to open a wave window go to 'File->New Wave
#   }
# }
# run all

=== MODE 0: CPOL=0, CPHA=0 ===
TX (Master->Slave): a5 | RX at Slave: a5
TX (Slave->Master): 3c | RX at Master: 3c
RESULT: PASS

=== MODE 1: CPOL=0, CPHA=1 ===
TX (Master->Slave): a5 | RX at Slave: a5
TX (Slave->Master): 3c | RX at Master: 3c
RESULT: PASS

=== MODE 2: CPOL=1, CPHA=0 ===
TX (Master->Slave): a5 | RX at Slave: a5
TX (Slave->Master): 3c | RX at Master: 3c
RESULT: PASS

=== MODE 3: CPOL=1, CPHA=1 ===
TX (Master->Slave): a5 | RX at Slave: a5
TX (Slave->Master): 3c | RX at Master: 3c
RESULT: PASS
$finish called at time : 3235 ns : File "C:/Users/smara/OneDrive/Desktop/vivado/spi master/spi master.srcs/sim_1/new/tb_spi_master.v" Line 158
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_spi_master_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for All
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak = 3352.250 ; gain = 0.000
```

Figure 8.1 Simulation Console Output Showing Successful SPI Transactions

Timing Diagram and Waveform Analysis

Timing diagrams were analyzed to verify correct SPI protocol operation, mode-dependent timing behavior, and transaction control. A combined simulation waveform was captured showing complete SPI transactions for all four SPI modes (Modes 0–3), from chip select assertion to deassertion.

The waveform includes the system clock, SPI clock (SCLK), chip select (CS), MOSI, MISO, transmit data, receive data, and internal bit counter signals. For each transaction, CS is asserted low before the start of clock activity and remains asserted for the entire 8-bit transfer, clearly defining a single SPI frame. CS is deasserted only after the final bit is sampled, confirming correct transaction boundary control.

The SPI clock idle level is observed to match the configured clock polarity (CPOL) for each mode. For CPHA = 0 operation, the first transmit data bit is preloaded on MOSI before the initial active clock edge, and data sampling occurs on the first clock transition. For CPHA = 1 operation, no preload occurs; data shifting begins only after the first clock edge, and sampling occurs on the subsequent edge. This behavior is consistently observed across both CPOL = 0 and CPOL = 1 configurations.

The internal bit counter increments once per sampled bit and reaches a count of seven at the completion of each byte transfer, confirming that exactly eight clock cycles are used per transaction. The received data at the master matches the transmitted slave data (0x3C), and the received data at the slave matches the transmitted master data (0xA5), demonstrating correct full-duplex operation.

Overall, the observed timing relationships between SCLK, MOSI, MISO, and CS for all four SPI modes match the SPI protocol specifications, confirming correct functional behavior and accurate mode-dependent timing implementation of the SPI Master controller.

Figure 5. SPI Timing Waveform Showing Complete Transactions Across All Four Modes (Modes 0–3)

CHALLENGES AND SOLUTIONS

Correct Handling of CPOL and CPHA Across All SPI Modes

A major challenge in the implementation was supporting all four SPI modes defined by the combinations of clock polarity (CPOL) and clock phase (CPHA). Each mode requires different timing relationships between the SPI clock edges and data shifting or sampling operations. Incorrect interpretation of these edges can lead to invalid data transfer.

This challenge was addressed by explicitly deriving separate shift and sample edges based on the CPOL and CPHA values. Internal detection of rising and falling edges of the SPI clock was implemented, and mode-dependent logic was used to ensure that data shifting and sampling occur on the correct clock edges for each SPI mode.

CPHA Preload Handling and First-Bit Alignment

Another critical challenge was ensuring correct first-bit timing, particularly for CPHA = 0 operation. In this mode, the first transmit data bit must be available on the MOSI line before the first active clock edge. Failing to preload the data correctly can cause the first bit to be missed by the slave.

This issue was resolved by introducing preload logic that places the most significant bit of the transmit data onto the MOSI line before the SPI clock starts toggling. A preload control flag was used to distinguish between initial data setup and subsequent shifting operations, ensuring protocol-compliant behavior for both CPHA = 0 and CPHA = 1 modes.

SPI Clock Generation and Idle State Control

Generating the SPI clock with correct timing and idle behavior posed another challenge. The SPI clock must remain at a defined idle level based on the CPOL setting when no transaction is active, and it must toggle only during an active transfer.

This was addressed by implementing an internal clock divider controlled by an enable signal from the FSM. When clock generation is disabled, the SPI clock is forced to the CPOL-

defined idle level. This approach ensures clean clock behavior, avoids spurious transitions, and maintains proper synchronization with data transfer operations.

Bit Count Control and Transaction Completion

Accurately tracking the number of transmitted and received bits during a transaction was another design challenge. Improper bit counting could result in premature termination or incomplete data transfer.

This challenge was solved by using a dedicated bit counter that increments only on valid sample edges after data preloading is complete. The FSM monitors the bit counter and transitions to the completion state only after the full data word has been successfully transferred.

Verification Across All SPI Modes

Verifying correct operation across all four SPI modes was a non-trivial task, as each mode exhibits different timing behavior. Manual waveform inspection alone was insufficient to guarantee correctness for bidirectional data transfer.

To address this, a self-checking testbench was developed with a behavioral SPI slave model capable of operating in all four modes. The testbench automatically compared transmitted and received data for both master and slave, and simulation console outputs were used to confirm successful operation across all modes.

CONCLUSION

This project successfully achieved the design and simulation-based verification of a 4-mode SPI Master controller using Verilog HDL. The controller supports all four SPI modes defined by clock polarity (CPOL) and clock phase (CPHA) and demonstrates correct protocol behavior for single-byte full-duplex communication.

The design employs an FSM-based control architecture, internal SPI clock generation, and mode-dependent data shifting and sampling logic to ensure protocol-accurate timing. Verification using a self-checking testbench and waveform-based analysis confirmed correct data transfer and timing behavior across all SPI modes. The results demonstrate that the design meets the stated objectives of functional correctness and reliable operation.

The outcomes of this project highlight the importance of careful timing analysis and verification in digital protocol controller design. By emphasizing correctness across multiple SPI configurations, the project provides a strong foundation for further development of SPI-based communication systems.

Future work may include extending the design to support multi-byte and continuous data transfers, parameterizing the clock divider for flexible SPI clock frequencies, supporting multiple slave devices, and integrating the controller with standard bus interfaces such as AXI or APB. Hardware implementation and testing on an FPGA platform can also be explored as an extension of this work.

REFERENCES

[1] Texas Instruments, “Serial Peripheral Interface (SPI) User Guide,”
Keystone Architecture, Literature No. SPRUGP2A, March 2012.

[2] I. Sengupta, “Hardware Modeling Using Verilog,”
NPTEL Online Course, Indian Institute of Technology Kharagpur.

[3] Xilinx Inc., “Vivado Design Suite User Guide: Logic Simulation,”
UG900, Xilinx.

[4] Motorola Inc., “SPI Block Guide,”
Application Note AN991, 1985.

[5] S. Palnitkar, “Verilog HDL: A Guide to Digital Design and Synthesis,”
2nd ed., Pearson Education.

APPENDICES

SPI Master RTL Code

This appendix presents the complete Verilog RTL implementation of the 4-mode SPI Master controller used in this project.

```
`timescale 1ns / 1ps


module spi_master (
    input  wire        clk,        // system clock
    input  wire        rst,        // synchronous reset
    input  wire        start,      // start transaction
    input  wire [7:0]  tx_data,    // data to send
    input wire        CPOL,        // clock idle polarity
    input wire        CPHA,        // clock phase

    output reg         mosi,
    input  wire        miso,
    output reg         sclk,
    output reg         cs,
    output reg         done,
    output reg [7:0]   rx_data
);

// FSM state encoding
    parameter IDLE       = 3'd0;
    parameter LOAD       = 3'd1;
    parameter ASSERT_CS  = 3'd2;
    parameter TRANSFER   = 3'd3;
```

```
    parameter DONE_STATE = 3'd4;

reg [2:0] state, next_state;

// Shift registers and bit counter
    reg [7:0] tx_shift;
    reg [7:0] rx_shift;
    reg [2:0] bit_cnt;

// sclk generation
reg sclk_en;
reg [1:0] clk_div_cnt;  // counts 0 to 3
always @(posedge clk) begin
    if (rst || !sclk_en) begin
        sclk <= CPOL;
        clk_div_cnt <= 2'd0;
    end else begin
        if (clk_div_cnt == 2'd3) begin
            clk_div_cnt <= 2'd0;
            sclk <= ~sclk;
        end else begin
            clk_div_cnt <= clk_div_cnt + 1;
        end
    end
end

// detect sclk edges
reg sclk_prev;
always @(posedge clk) begin
```

```
    if (rst)
          sclk_prev <= CPOL;
    else
        sclk_prev <= sclk;
end

wire sclk_rising;
wire sclk_falling;

assign sclk_rising  = (sclk_prev == 1'b0 && sclk == 1'b1);
assign sclk_falling = (sclk_prev == 1'b1 && sclk == 1'b0);

wire shift_edge;
wire sample_edge;

assign shift_edge  =
    (CPOL == 0 && CPHA == 0) ? sclk_falling :
    (CPOL == 0 && CPHA == 1) ? sclk_rising  :
    (CPOL == 1 && CPHA == 0) ? sclk_rising  :
                               sclk_falling ;

assign sample_edge =
    (CPOL == 0 && CPHA == 0) ? sclk_rising  :
    (CPOL == 0 && CPHA == 1) ? sclk_falling :
    (CPOL == 1 && CPHA == 0) ? sclk_falling :
                               sclk_rising  ;

// FSM: state register
always @(posedge clk) begin
```

```
        if (rst)
            state <= IDLE;
        else
            state <= next_state;
    end

// FSM: next-state logic
always @(*) begin
        next_state = state;
        case (state)
            IDLE: begin
                if (start)
                    next_state = LOAD;
            end

            LOAD: begin
                next_state = ASSERT_CS;
            end

            ASSERT_CS: begin
                next_state = TRANSFER;
            end

            TRANSFER: begin
                if (sample_edge && bit_cnt == 3'd7)
                    next_state = DONE_STATE;
            end

            DONE_STATE: begin
```

```
                next_state = IDLE;
            end

            default: next_state = IDLE;
        endcase
    end

reg preload_done;

// FSM: output and datapath logic
always @(posedge clk) begin
        if (rst) begin
            cs       <= 1'b1;
            done     <= 1'b0;
            sclk_en  <= 1'b0;
            bit_cnt  <= 3'd0;
            tx_shift <= 8'd0;
            rx_shift <= 8'd0;
            rx_data  <= 8'd0;
            mosi     <= 1'b0;
        end else begin
            done <= 1'b0;

            case (state)

                IDLE: begin
                    cs      <= 1'b1;
                    sclk_en <= 1'b0;
                    bit_cnt <= 3'd0;
```

```
end

LOAD: begin
    tx_shift <= tx_data;   // latch TX data once
end

ASSERT_CS: begin
    cs      <= 1'b0;
    sclk_en <= 1'b1;
    preload_done <= 1'b0;

    if (CPHA == 0) begin
        mosi    <= tx_shift[7];
        preload_done <= 1'b1;
    end
end

TRANSFER: begin

    // SHIFT (TX)
    if (shift_edge) begin
        if (CPHA == 0 || preload_done) begin
            tx_shift <= {tx_shift[6:0], 1'b0};
            mosi     <= tx_shift[6];
        end else begin
            // First shift edge in CPHA=1: preload instead
            mosi <= tx_shift[7];
            preload_done <= 1'b1;
        end
```

```
                end


                // SAMPLE (RX)
                if (sample_edge) begin
                    if (preload_done) begin
                        rx_shift <= {rx_shift[6:0], miso};
                        bit_cnt  <= bit_cnt + 1'b1;
                    end
                end
            end


            DONE_STATE: begin
                sclk_en <= 1'b0;
                cs      <= 1'b1;
                rx_data <= rx_shift;
                done    <= 1'b1;
            end


        endcase
    end
end


endmodule
```

SPI Master Testbench Code

This appendix presents the complete Verilog testbench used for functional verification of the SPI Master controller. The testbench includes a behavioral SPI slave model and automated PASS/FAIL checking for all four SPI modes.

```
`timescale 1ns / 1ps

module tb_spi_master;

// Testbench signals
reg clk;
reg rst;
reg start;
reg [7:0] tx_data;
reg CPOL;
reg CPHA;
wire mosi;
reg miso;
wire sclk;
wire cs;
wire done;
wire [7:0] rx_data;

// DUT instantiation
spi_master dut (
    .clk(clk),
    .rst(rst),
    .start(start),
    .tx_data(tx_data),
    .CPOL(CPOL),
    .CPHA(CPHA),
    .mosi(mosi),
    .miso(miso),
    .sclk(sclk),
```

```
    .cs(cs),
    .done(done),
    .rx_data(rx_data)
);

// Clock generation (100 MHz)
always #5 clk = ~clk; // 10 ns period

// SPI Slave Model - handles all 4 modes
reg [7:0] slave_data;
reg slave_preload_done;

reg [7:0] slave_rx_shift;
reg [2:0] slave_bit_cnt;

// Slave initialization on CS falling edge
always @(negedge cs) begin
    slave_data = 8'h3C;
    slave_preload_done = 1'b0;
    if (CPHA == 0) begin
        miso = slave_data[7];
        slave_preload_done = 1'b1;
      end
end

reg [7:0] slave_rx_data;

always @(posedge cs) begin
    slave_rx_data <= slave_rx_shift;
```

```
end

// Mode 0 & 3: shift on falling edge
always @(negedge sclk) begin
    if (!cs && ((CPOL == 0 && CPHA == 0) || (CPOL == 1 && CPHA == 1))) begin
        if (CPHA == 0 || slave_preload_done) begin
            slave_data <= {slave_data[6:0], 1'b0};
            miso <= slave_data[6];
        end else begin
            miso <= slave_data[7];
            slave_preload_done <= 1'b1;
        end
    end
end

// Mode 1 & 2: shift on rising edge
always @(posedge sclk) begin
    if (!cs && ((CPOL == 0 && CPHA == 1) || (CPOL == 1 && CPHA == 0))) begin
        if (CPHA == 0 || slave_preload_done) begin
            slave_data <= {slave_data[6:0], 1'b0};
            miso <= slave_data[6];
        end else begin
            miso <= slave_data[7];
            slave_preload_done <= 1'b1;
        end
    end
end

// SLAVE RX (SAMPLE MOSI)
```

```
// Modes 0 & 3 → sample on rising edge
    always @(posedge sclk) begin
        if (!cs && ((CPOL == 0 && CPHA == 0) || (CPOL == 1 && CPHA == 1))) begin
            slave_rx_shift <= {slave_rx_shift[6:0], mosi};
            slave_bit_cnt <= slave_bit_cnt + 1'b1;
        end
    end

// Modes 1 & 2 → sample on falling edge
    always @(negedge sclk) begin
        if (!cs && ((CPOL == 0 && CPHA == 1) || (CPOL == 1 && CPHA == 0))) begin
            slave_rx_shift <= {slave_rx_shift[6:0], mosi};
            slave_bit_cnt <= slave_bit_cnt + 1'b1;
        end
    end

initial begin
    clk   = 0;
    rst   = 1;
    start = 0;
    tx_data = 8'hA5;
    miso = 0;

    #50 rst = 0;

    // MODE 0 (CPOL=0, CPHA=0)
    #20;
    CPOL = 0; CPHA = 0;
```

```
    $display("\n=== MODE 0: CPOL=%b, CPHA=%b ===", CPOL, CPHA);
    run_and_check();

    // MODE 1 (CPOL=0, CPHA=1)
    #50;  // Add delay between modes
    CPOL = 0; CPHA = 1;
    $display("\n=== MODE 1: CPOL=%b, CPHA=%b ===", CPOL, CPHA);
    run_and_check();

    // MODE 2 (CPOL=1, CPHA=0)
    #50;
    CPOL = 1; CPHA = 0;
    $display("\n=== MODE 2: CPOL=%b, CPHA=%b ===", CPOL, CPHA);
    run_and_check();

    // MODE 3 (CPOL=1, CPHA=1)
    #50;
    CPOL = 1; CPHA = 1;
    $display("\n=== MODE 3: CPOL=%b, CPHA=%b ===", CPOL, CPHA);
    run_and_check();

    #100 $finish;
end

task run_and_check;
    reg [7:0] master_rx;
    reg [7:0] slave_rx;
    begin
        // RESET SLAVE SIDE STATE
```

```
slave_rx_shift = 8'd0;
slave_bit_cnt  = 3'd0;

// START TRANSACTION
@(negedge clk);
start = 1'b1;
@(negedge clk);
start = 1'b0;

// WAIT FOR CS ASSERT
wait (cs == 1'b0);

// WAIT FOR CS DEASSERT (transaction truly finished)
wait (cs == 1'b1);

@(posedge clk);

master_rx = rx_data;
slave_rx  = slave_rx_data;

$display("TX (Master->Slave): %h | RX at Slave: %h",
         tx_data, slave_rx);
$display("TX (Slave->Master): %h | RX at Master: %h",
         8'h3C, master_rx);

if (master_rx === 8'h3C && slave_rx === tx_data) begin
    $display("RESULT: PASS");
end
else begin
```

```
            $display("RESULT: FAIL");
            if (master_rx !== 8'h3C)
                $display("  ERROR: Master RX mismatch");
            if (slave_rx !== tx_data)
                $display("  ERROR: Slave RX mismatch");
        end

        repeat (5) @(posedge clk);
    end
endtask

endmodule
```